C++ Programming Problem Set

Module: Strings (C-Strings, std::string & Stream Input Mechanics)

CS-SIMPLIFIED

Author: Gaurav Kandpal

Target Concept: Character Arrays & STL std::string

Integrates: Streams, Loops, Functions, References, Arrays & Memory

Instructions & Pedagogical Guidelines

This problem set focuses on string processing in C++. You will solve problems spanning raw C-style character arrays with null terminators (`'\0'`), modern STL `std::string` containers, stream flushing traps (`cin.ignore()`), pass-by-const-reference efficiency, substring extraction, and full sentence parsing. Ensure all inputs are properly sanitized before processing.

Level 1: C-Strings, Input Streams & Stream Buffer Flushing

Problem 1: C-String Null-Terminator & Memory Bounds Inspector

C-Style Char Array | Null Terminator | Memory

Declare a raw character array `char str[10] = "Gaurav";`. Write a program using loop iteration to output every character alongside its ASCII integer value and physical RAM address. Deliberately overwrite the 6th index slot `str[6] = 'X'` (removing the null terminator `'\0'`), and attempt to print `str` via `std::cout` to demonstrate reading past buffer bounds (garbage characters or crash).

Null Terminator ('\0') | Contiguous RAM Offset | C-String Buffer Overrun

Sample Setup	Observed Memory State
`char str[10] = "Gaurav";`	`Length (strlen): 6 | Byte size (sizeof): 10` `Removing '\0' at str[6] triggers illegal array boundary read.`

Problem 2: The Stranded Buffer Fix (cin >> vs getline)

Input Streams | Buffer Flushing | Streams

Build a user registration tool prompting for an integer ID, user age, and full name. Prompt the user for numerical age using `cin >> age`, followed immediately by `std::getline(cin, fullName)`. Show how unhandled newline characters (`'\n'`) left in the input buffer cause `getline` to skip prompt execution. Implement the correct solution using `cin.ignore(numeric_limits<streamsize>::max(), '\n')`.

cin Stream Delimiters | cin.ignore() Flushing | getline Whitespace Handling

Sample Input Sequence	Corrected Program Output
`Age: 21` `Name: Gaurav Kandpal`	`Age: 21 | Full Name: "Gaurav Kandpal"` `Buffer flushed successfully; prompt executed cleanly.`

Problem 3: In-Place Case Converter & Non-Alphabetical Filter

Pass-by-Reference | Character Functions | Modularity

Write a modular function `void transformText(std::string &str, bool toUpper)` that modifies a reference string in-place. If `toUpper` is true, convert all lowercase letters to uppercase; otherwise convert to lowercase. Simultaneously replace all numerical digits (0-9) with an asterisk `'*'`. Test the function using inputs in `main()`.

Pass-by-Reference String | std::toupper / std::isdigit | In-Place String Mutation

Sample Input String	Transformed Output
`str = "Code2026", toUpper = true`	`Modified str: "CODE****"`

Level 2: String Searching, Substrings & Multi-String Processing

Problem 4: Palindrome Inspector & Sanitizer

Two-Pointer | String Searching | Loops

Write a function `bool isPalindrome(const std::string &s)` that evaluates whether a sentence is a palindrome. The function must ignore spaces, punctuation, and letter casing. Use a two-pointer approach (`left = 0`, `right = s.length() - 1`) iterating inward toward the center to check symmetry without creating unnecessary temporary copies.

```
// Two-pointer symmetry algorithm with const reference protection: bool isPalindrome(const std::string &s) { int left = 0, right = s.length() - 1; // Skip non-alphanumeric chars and compare case-insensitively... }
```

Sample Sentence	Symmetry Evaluation
`"A man, a plan, a canal: Panama"`	`Cleaned: "amanaplanacanalpanama"` `Result: PALINDROME (TRUE)`

Problem 5: Substring Search & Replace Engine

std::string Member Functions | find() & substr()

Implement a function `int replaceAllOccurrences(std::string &target, const std::string &findStr, const std::string &replaceStr)`. The function searches for every instance of `findStr` inside `target` using `target.find()`, replaces it with `replaceStr` using `target.replace()`, and returns the total replacement count. Ensure the search loop advances correctly past substituted segments to avoid infinite loops.

std::string::find() | std::string::npos | std::string::replace()

Initial Target & Parameters	Replacement Output
`target = "the cat sat on the mat", find = "the", replace = "a"`	`Total Replaced: 2` `Modified target: "a cat sat on a mat"`

Problem 6: Array of Strings Sort & Lexicographical Search

String Arrays | Sorting Algorithms | Pass-by-Address

Create a program that reads N student names into an array of strings `std::string names[10]`. Write two modular functions:
1. `void sortNames(std::string arr[], int size)` using bubble sort or selection sort with lexicographical string comparisons (> operator).
2. `int findName(const std::string arr[], int size, const std::string &query)` using binary search on the sorted array.

Array of std::string | Lexicographical Comparison | Binary Search on Strings

Input Array	Sorted & Search Result
`Unsorted: ["Zara", "Alice", "Bob"]`	`Sorted: ["Alice", "Bob", "Zara"]` `Search Query "Bob": Found at Index 1`

Level 3: Advanced Parsing, SSO Overhead & Capstone Project

Problem 7: Word Frequency Counter & Stream Tokenizer

std::stringstream | Tokenization | Data Aggregation

Write a program that takes a complete paragraph using `getline`. Tokenize the sentence into individual word tokens using `std::stringstream`. Normalize each word by stripping leading/trailing punctuation and converting letters to lowercase. Store the tokens in an array of structures or dynamic vector to count unique word occurrences and display a frequency table.

```
// Tokenizing sentences into words via stringstream: std::stringstream ss(paragraph); std::string token; while (ss >> token) { // Sanitize token and record word count... }
```

Sample Paragraph	Token Frequency Output

`"C++ is fast. C++ is powerful!"`	`Word "c++": 2`
	`Word "is": 2`
	`Word "fast": 1 | Word "powerful": 1`

Problem 8: Const Reference Performance vs SSO Heap Allocation

Const Reference | Small String Optimization | Efficiency

In modern C++, short strings (<= 15 chars) fit on the Stack via Small String Optimization (SSO), while long strings force dynamic Heap memory allocation. Write a benchmark program comparing function call passing methods. Implement `void passByValue(std::string s)` and `void passByConstRef(const std::string &s)`. Pass a 100-character string in a loop running 100,000 times to highlight the performance advantage of avoiding heap allocation copies.

Small String Optimization (SSO) | Heap Buffer Allocation | Pass-by-Const-Reference

Passing Strategy	Heap Allocation & Copy Overhead
`passByValue(longStr)`	`High Cost: Triggers dynamic heap allocation copy for every call.`
`passByConstRef(longStr)`	`Zero Copy: Passes pointer handle; avoids heap allocations entirely.`

Problem 9: Cumulative Capstone: CSV Log File Record Parser

Comprehensive Integration | Parsing | Structures & Strings

Build a complete CSV log record parsing engine. Define a structure `UserRecord` containing `id` (int), `name` (string), and `score` (double). Implement functions:
1. `bool parseCSVLine(const std::string &line, UserRecord &outRecord)` — Parses a line structured as `"101,Gaurav Kandpal,94.5"` using `find()` and `substr()`.
2. `void generateSummary(const UserRecord records[], int size)` — Computes class average score and prints a formatted summary table.
Process an array of CSV string records in `main()`, handling malformed lines safely.

CSV String Parsing | std::stoi & std::stod Conversion | Struct Reference Output | Complete Module Capstone

Raw CSV Line Inputs	Parsed Struct Record Output

`"101,Gaurav Kandpal,94.5"`
`"102,Alice Smith,88.0"`
	`[ID: 101] Name: Gaurav Kandpal | Score: 94.5`
	`[ID: 102] Name: Alice Smith | Score: 88.0`
	`Class Average: 91.25`